SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE

COURSE CODE: CSA1221

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I I. Leela Sravan Kumar (192425272) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Answer:

Problem Analysis:

In a smart city surveillance system, multiple cameras continuously send video data to the computer. The I/O devices (cameras) transfer the video to main memory (RAM)**,** and the CPU fetches this data from memory to detect suspicious activities. During peak hours, thousands of video frames are processed simultaneously, causing delays.

Interaction of CPU, Memory, and I/O:

- I/O Devices**:** Capture and send video streams to memory.
- Memory (RAM): Temporarily stores the video frames.
- CPU: Fetches frames from memory, processes them using AI algorithms, and generates alerts.
- Large data transfers increase memory traffic.
- CPU waits for memory access.
- I/O devices compete for the same communication path.
- This results in lag and delayed threat detection.

Bus Structure Comparison

Single-Bus Architecture	Multi-Bus Architecture
One bus is shared by CPU, memory, and I/O devices.	Separate buses are available for CPU, memory, and I/O.
Only one data transfer occurs at a time.	Multiple transfers occur simultaneously.
High traffic causes bottlenecks and delays.	Reduces congestion and improves performance.
Suitable for small systems.	Suitable for high-speed surveillance systems.

Improving the Instruction Execution Cycle

The CPU executes instructions using the **Fetch → Decode → Execute** cycle.
Performance can be improved by:

- Instruction Pipelining – Executes multiple instructions simultaneously.
- Cache Memory – Reduces memory access time.
- DMA (Direct Memory Access) – Allows I/O devices to transfer data directly to memory without CPU intervention.
- Branch Prediction – Reduces execution delays caused by conditional instructions.
- Multi-core Processors – Different cores process multiple camera feeds in parallel.

Conclusion

A multi-bus architecture, along with cache memory, DMA, pipelining, and multi-core CPUs, significantly improves data transfer speed and reduces CPU waiting time. These enhancements minimize lag and enable faster, real-time threat detection in smart city surveillance systems.

Result: The surveillance system processes multiple video streams efficiently, improving response time, reliability, and public safety.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode– execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Answer

Problem Analysis:

A real-time stock trading application processes millions of buy and sell transactions every second. During high trading volume, the processor experiences delays because it spends more clock cycles accessing memory and executing branch instructions. This increases the Cycles Per Instruction (CPI), resulting in slower transaction processing.

Impact of the Fetch–Decode–Execute Cycle:

The CPU performs every instruction in three stages:

- Fetch: Retrieves the instruction from memory.
- Decode: Interprets the instruction and determines the required operation.
- Execute: Performs the operation and stores the result.
- Frequent memory access slows the Fetch stage.
- Complex branch instructions increase the Decode stage time.
- The Execute stage waits for previous instructions to complete.
- As a result, the execution time increases and users experience transaction delays.

Methods to Reduce CPI:

To improve CPU performance, the following architectural enhancements can be used:

- Cache Memory: Stores frequently used data and instructions, reducing memory access time.
- Instruction Pipelining: Executes multiple instructions simultaneously, increasing throughput.
- Branch Prediction**:** Predicts the outcome of branch instructions, minimizing pipeline stalls.
- Out-of-Order Execution: Executes independent instructions without waiting for previous ones.
- Multi-Core Processors: Distributes trading transactions across multiple CPU cores.
- High-Speed RAM: Reduces memory latency and improves instruction fetch speed.

Result

These improvements reduce the CPI, decrease execution time, and allow the processor to handle more transactions per second with lower latency.

Conclusion

By using cache memory, instruction pipelining, branch prediction, out-of-order execution, multi-core processors, and faster memory, the stock trading application achieves lower CPI and faster execution. This ensures quick transaction processing, improves system performance, and provides a better user experience during peak trading hours.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

Answer

1. Problem Analysis:

In an industrial temperature control system, the 8085 microprocessor receives temperature values from sensors, processes them, and controls actuators such as heaters or cooling fans. If the program uses an incorrect addressing mode, the processor may read data from the wrong memory location, resulting in incorrect temperature readings and improper system operation.

2. Influence of Addressing Modes on Data Access

- Immediate Addressing: Data is provided directly in the instruction (e.g., MVI A, 25H).
- Register Addressing: Data is stored in CPU registers (e.g., MOV A, B).
- Direct Addressing: Data is accessed from a specified memory location (e.g., LDA 2500H).
- Register Indirect Addressing: The memory address is stored in a register pair such as HL (e.g., MOV A, M).
- Implied Addressing: The operand is implied by the instruction (e.g., CMA).

Choosing the correct addressing mode ensures that sensor data is read from the correct location.

3. Possible Causes of Errors

- Incorrect memory address used for sensor data.
- Wrong register or register pair selected.
- Improper initialization of memory locations.
- Programming mistakes in data transfer instructions.
- Noise or invalid sensor values not checked before processing.

4. Using the 8085 Instruction Set Effectively

To improve accuracy, the following instructions can be used:

- LDA – Load sensor data from memory.
- STA – Store processed data.
- MOV – Transfer data between registers.
- MVI – Initialize registers with required values.
- IN – Read data from sensor input ports.
- OUT – Send control signals to actuators.
- CMP – Compare temperature with the reference value.
- JZ, JC, JNC – Perform conditional decisions for controlling heater or cooler.

5. Conclusion

By selecting the correct addressing mode, properly initializing memory and registers, and using the 8085 instruction set correctly, the embedded temperature control system can process accurate sensor readings, control actuators reliably, and improve the overall safety and efficiency of industrial operations.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

Answer

1. Problem Analysis

An 8086-based computing system executes multiple programs using segmented memory. Each program stores its instructions, data, and stack in separate memory segments. If the segment registers are not initialized correctly, the CPU may access the wrong memory location, causing incorrect data access and stack overflow errors.

2. Working of Segmentation in 8086

The **8086** divides memory into four segments:

- Code Segment (CS)**:** Stores program instructions and is accessed using the Instruction Pointer (IP)**.**
- Data Segment (DS)**:** Stores variables and data used by the program.
- Stack Segment (SS)**:** Stores return addresses, local variables, and temporary data using the Stack Pointer **(SP)**.
    - Extra Segment (ES): Used for additional data storage and string operations.

This segmentation allows multiple programs to run efficiently by organizing memory into separate sections.

3. Problems Caused by Improper Segment Handling

- Incorrect values in CS, DS, or SS cause the CPU to access the wrong memory location.
- Improper PUSH and POP operations can lead to stack overflow or stack corruption.
- Wrong addressing modes may fetch incorrect data from memory.
- Failure to initialize segment registers before execution causes program errors and crashes.

4. Managing Instruction Execution and Addressing Modes

- Initialize CS, DS, SS, and ES before running the program.
- Use PUSH and POP instructions correctly to maintain the stack.
- Select appropriate addressing modes (Direct, Register, Based, Indexed, Based-Indexed) for accurate data access.
- Ensure the Fetch–Decode–Execute cycle accesses the correct segment and memory location.
- Validate memory references before execution to avoid invalid access.

5. Conclusion

Proper management of memory segmentation**,** segment registers**,** instruction execution, and addressing modes ensures accurate data access, prevents stack overflow, and improves the reliability and performance of an 8086-based multitasking system.

5. **A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.**

Answer

1. Problem Analysis

In a data communication system, packet data is commonly represented in hexadecimal because it is compact and easy for programmers to read. However, the CPU processes all data in binary. If hexadecimal values are

converted incorrectly into binary, the processor may execute incorrect instructions, resulting in communication errors and data corruption.

2. Importance of Hexadecimal to Binary Conversion

- Hexadecimal is a human-readable representation of binary data.
- Every 1 hexadecimal digit = 4 binary bits.
- Correct conversion ensures that the CPU receives the exact binary instructions for execution.
- It simplifies debugging, memory addressing, and packet analysis.

Example:

- Hexadecimal 3A = Binary 00111010
- If converted incorrectly, the CPU may interpret a different instruction or data value.

3. Effects of Conversion Errors

- Wrong packet information to be processed.
- Execution of incorrect CPU instructions.
- Invalid memory addresses.
- Communication failures between devices.
- Data corruption and system crashes.
- Reduced reliability in real-time communication systems.

4. Role of Efficient CPU Design and Benchmarking

- Error Detection Techniques (parity bits, checksums, CRC) verify data accuracy.
- Efficient CPU Design with fast instruction decoding and error handling improves reliability.
- Benchmarking measures CPU performance, execution time, and error rates under different workloads.
- Validation and Debugging Tools help detect incorrect conversions before deployment.
- Proper software testing ensures accurate hexadecimal-to-binary conversion.

5. Conclusion

Correct hexadecimal-to-binary conversion is essential for accurate instruction execution and reliable packet processing. Efficient CPU architecture, benchmarking, and error detection techniques help identify conversion mistakes, reduce processing errors, and improve the performance of real-time data communication systems.